

Aqui está o planejamento slide por slide para os **11 slides** da apresentação do seu grupo (**Grupo 4**), seguindo rigorosamente a estrutura mínima exigida pelo Guia do Aluno.

Cada slide foi desenhado para ser visualmente limpo (usando tópicos rápidos, sem textos gigantes) e acompanha uma **orientação de fala** para ajudar os integrantes na hora de apresentar nos 10 minutos de limite.

Slide 1: Capa

- **Título Principal:** Relacionamentos entre Objetos & POO Aplicada ao Back-End
 - **Subtítulo:** Organizando dados, regras e ciclos de vida no servidor
 - **Identificação:**
 - Nome dos integrantes (no máximo 5 alunos)
 - Curso Técnico em Desenvolvimento de Sistemas
 - Unidade Curricular: Programação Back-End
 -  **O que falar:** *Dê as boas-vindas à turma e ao professor. Apresente o grupo e explique que hoje vocês vão mostrar como a Programação Orientada a Objetos é usada para modelar conexões do mundo real e organizar a arquitetura de um servidor.*
-

Slide 2: Visão Geral

- **Título:** O que Vamos Explicar Hoje?
- **Tópicos em Bullet Points:**
 - As conexões entre objetos: Associação, Agregação e Composição.
 - A diferença crucial: Ciclos de vida e dependência de dados.
 - Modelagem no Back-End: Entidades de Domínio e Classes de Serviço.
 - Demonstração prática de um e-commerce em JavaScript.
-  **O que falar:** *Faça uma introdução rápida dos tópicos do slide. Explique que o seminário está dividido em entender a teoria das conexões e, depois, ver como aplicamos isso na prática em uma aplicação que roda no servidor.*

Slide 3: Primeiro Conceito: Relacionamentos

- **Título:** Associação, Agregação e Composição
- **Texto Visual:**
 - **Associação:** Conexão simples e livre. Os objetos são completamente independentes.
 - **Agregação:** Relação "todo-parte" onde a parte pode existir separadamente (relação fraca).
 - **Composição:** Relação "todo-parte" onde a parte pertence rigidamente ao todo e não tem vida independente (relação forte).
-  **O que falar:** *Explique as definições técnicas de cada uma de forma pausada. Enfatize que, embora pareçam similares, a forma como os objetos dependem uns dos outros muda bastante.*

Slide 4: Segundo Conceito: POO Aplicada ao Back-End

- **Título:** Organização do Servidor: Entidades e Serviços
- **Texto Visual:**
 - **Entidades do Domínio:** Classes que modelam conceitos reais do negócio e guardam os dados e estados do sistema (ex: Usuario, Produto, Pedido).
 - **Classes de Serviço (Services):** Classes que não representam dados, mas concentram comportamentos, lógica e regras de negócio complexas (ex: PedidoService, UsuarioService).
-  **O que falar:** *Mostre que o Back-End separa os "dados" das "ações". As Entidades representam as informações brutas (que depois vão para o banco de dados), enquanto os Serviços ditam as regras (validação, segurança, cálculos).*

Slide 5: Como os Conceitos se Relacionam

- **Título:** A Conexão: Modelando Bancos de Dados e Regras
- **Texto Visual:**

- As conexões entre os objetos na memória simulam os dados armazenados no servidor:
 - **Associações:** Geram chaves estrangeiras (clienteld conecta um pedido a um cliente).
 - **Composições:** Definem exclusões em cascata (se um Pedido for excluído, os itens vinculados a ele devem ser excluídos automaticamente).
- As **Classes de Serviço** manipulam essas regras para processar requisições com segurança.
-  **O que falar:** *Aqui você ganha pontos de integração! Explique que entender relacionamentos é crucial para saber como salvar dados no banco. Um Item do Pedido não pode ficar "órfão" no banco de dados se o pedido correspondente for deletado.*

Slide 6: Exemplo do Cotidiano (Analogia)

- **Título:** Analogia dos Relacionamentos
- **Texto Visual (ou diagrama simples):**
 - **Agregação: Time → Jogadores**
 - Se o Time for desativado no sistema, os Jogadores continuam existindo de forma independente no banco de dados da federação.
 - **Composição: Livro Físico → Páginas**
 - Se o Livro for totalmente destruído, as Páginas físicas dele deixam de existir junto com ele; elas não têm vida útil sozinhas.
-  **O que falar:** *Use este slide para garantir que a turma inteira entenda o conceito abstrato de forma fácil antes de mostrar o código. Use a analogia do time e dos jogadores e a do livro.*

Slide 7: Exemplo Aplicado à Programação

- **Título:** Nosso Cenário: Cliente, Pedido e ItemPedido
- **Texto Visual:**

- Cliente → **Associação** com Pedido. Estão ligados, mas o Cliente existe sem o Pedido, e vice-versa.
 - Pedido → **Composição** com ItemPedido. O Pedido gerencia o ciclo de vida dos itens diretamente.
 - PedidoService → Classe que recebe a requisição de compra, valida se há estoque e salva o Pedido.
 -  **O que falar:** *Mostre como aquele cenário de e-commerce real que todo mundo usa no dia a dia é traduzido para classes antes de abrir a tela de código.*
-

Slide 8: Trecho de Código

- **Título:** Implementação Prática em JavaScript
- **Texto Visual (Código Grande e Legível):**

```
class ItemPedido {  
  constructor(nome, preco) {  
    this.nome = nome;  
    this.preco = preco;  
  }  
}
```

```
class Pedido {  
  constructor(id, clienteld) {  
    this.id = id;  
    this.clienteld = clienteld; // Associação  
    this.itens = []; // Composição  
  }  
}
```

```
adicionarItem(nome, preco) {  
  // A criação do ItemPedido ocorre dentro do Pedido
```

```
this.itens.push(new ItemPedido(nome, preco));  
  
}  
  
}
```

-  **O que falar:** Não se apavore. Mostre que o código é curto e direto. Explique que criamos duas classes básicas que guardam dados (Entidades do Domínio).
-

Slide 9: Explicação do Código e Relação com Back-End

- **Título:** Como esse código opera no Servidor?
 - **Texto Visual:**
 - O papel do **this**: Aponta para o objeto atual em execução, garantindo que o pedido do Cliente A não misture itens com o do Cliente B.
 - Onde está a **Composição**: No método adicionarItem(), onde o próprio Pedido cria (new) a instância de ItemPedido. Se o pedido for destruído, a lista this.itens some junto.
 - Onde está a **Associação**: No atributo clienteId, ligando o pedido ao cliente apenas por uma referência.
 -  **O que falar:** Destaque o uso do "this" como forma de o objeto se referenciar. Explique que o operador "new" sendo chamado dentro de "Pedido" é o que consolida fisicamente a composição.
-

Slide 10: Conclusão

- **Título:** O que Devemos Lembrar?
- **Texto Visual:**
 - Sistemas Back-End de verdade dependem de modelagem precisa para evitar inconsistência de dados órfãos.
 - A divisão em **Entidades** (dados) e **Serviços** (comportamentos) mantém o servidor limpo e escalável.
 - Diferenciar **Agregação** de **Composição** dita as regras de exclusão no Banco de Dados.

-  **O que falar:** *Feche a apresentação reforçando que a POO não serve apenas para programar em si, mas sim para simular regras reais de negócios no código do servidor de maneira profissional e segura.*
-

Slide 11: Fontes Consultadas

- **Título:** Fontes e Referências
- **Texto Visual:**
 - **MDN Web Docs:** *Classes in JavaScript - Learn web development*
 - **MDN Web Docs:** *O básico sobre objetos JavaScript - Aprendendo desenvolvimento web*
 - **MDN Web Docs:** *Private elements - JavaScript*
 - Materiais Didáticos de Apoio ao Aluno (PDFs 01 e 02).
-  **O que falar:** *Agradeça a atenção de todos e do professor, mencione que as fontes consultadas garantiram a precisão técnica da sua pesquisa e abra espaço para dúvidas da classe.*